



Building Reliable AI Agents

From an alert to an incident dossier, using Python and Gemini.

Build an agent loop, then test its behavior.

- | | | |
|-----------|--|---------------|
| 01 | Understand tools and the agent loop
Follow one example from the alert to the first observation. | 10 min |
| 02 | Build in the notebook
Open the notebook, then complete checkpoints 1–3. | 31 min |
| 03 | Trigger failures and test the safeguards
Checkpoint 4: compare actual and expected behavior. | 5 min |
| 04 | Take the dossier home and explain your work
Find the evidence and suggest a test for your own field. | 4 min |

What tasks do you give an AI application today?

Your first uses

Drafting a message, explaining code, rephrasing an idea... What did you first use it for?

What you do today

What tasks do you give it now? What information and tools does it need to carry them out?

To investigate an incident that is happening now, the system needs access to its current state.

09:42: a clinic reports a temperature alert.

Alert CC-204

Clinic KCARE-ADJ-01

12.4 °C for 52 min

Vaccines · batch VX-204

The clinic's refrigerator has triggered an alert. Your team must check the readings and find the relevant procedure.

You are the team on duty.

Build an agent that prepares the incident dossier and requests an operator's decision.

Fictional exercise: simulated data, procedures and operator. No medical or physical action is performed.

Who does what when we investigate this alert?

The LLM proposes

An LLM is a language model. It uses its context to generate text or a tool call. Here, it requests the clinic's readings.

The application executes

Our Python code supplies tools to the model, checks the arguments and calls the function that reads the sensor data.

The agent continues

The agent is the application pursuing the goal: the model receives the tool result and chooses what comes next, until an answer or a stop.

A fixed workflow programs the steps. Here, the model chooses some of them based on the results.

The model requests a reading. Python retrieves it.

1. The model's proposed call

```
{  
  "name": "get_clinic_status",  
  "arguments": {  
    "clinic_id": "KCARE-ADJ-01"  
  }  
}
```

The name selects the function; the arguments identify the clinic. Python validates them before execution.

2. The tool's returned result

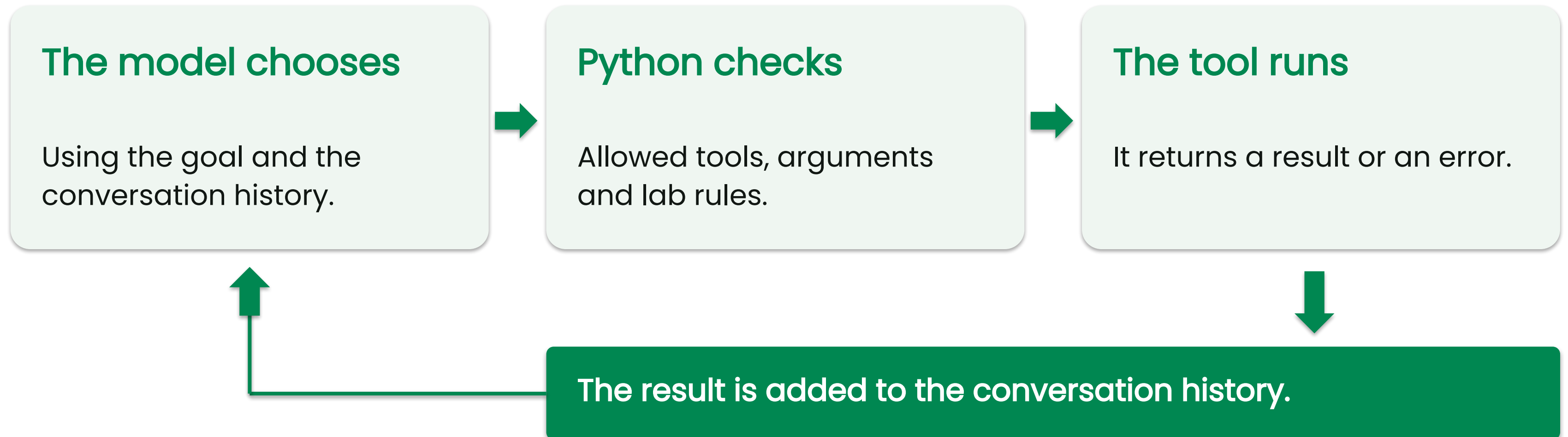
```
{  
  "clinic_id": "KCARE-ADJ-01",  
  "temperature_c": 12.4,  
  "excursion_minutes": 52,  
  "sensor_status": "OK"  
}
```

This excerpt contains the retrieved reading. We preserve the actual output, including any errors.

Normalized call and shortened result from the simulated scenario. The model receives the tool schemas.

Each result informs the next decision.

On each turn, the model proposes a tool call or an answer.



The model receives this observation on its next turn.

If the model proposes an answer, Python checks the evidence and either returns or blocks it.

We have the reading. What do we still need?

Moment	What is exchanged	What it enables
Turn 1	get_clinic_status clinic_id: KCARE-ADJ-01	Retrieve the clinic's state.
Observation	12.4 °C · 52 min · sensor OK	Keep the facts in the history.
Turn 2	search_cold_chain_sop query: critical temperature excursion	Find the relevant procedure.

Without the returned result, the model cannot know what the first tool found.

Five functions to investigate the alert.

Question	Python tool	Expected result
What does the sensor report?	<code>get_clinic_status</code>	The clinic's telemetry.
Which procedure applies?	<code>search_cold_chain_sop</code>	A retrieved local procedure.
What do the lab rules say?	<code>assess_excursion_risk</code>	Severity and proposed action.
How do we record the alert?	<code>create_incident</code>	An incident identifier.
Does the operator approve?	<code>request_human_review</code>	A decision for this incident.

The request determines the path. Authorization rules stay in code.

Three failures our code needs to recognize.

The reading changes

The tool returns 12.4 °C. The model then passes 5 °C to the risk calculation. Code must compare that value with the retrieved reading.

The call repeats

The model requests exactly the same reading again. Code detects an already executed call and stops the loop.

Approval is missing

The model wants to conclude before the required review. Code requires approval for the correct incident and action.

In the notebook, you will trigger these failures and find where the system stops.

Turn the alert into a request for the first reading.

1

Predict the first call.

Which function can tell us the state of KCARE-ADJ-01?

2

Complete TODO 1–2.

Give the model the history and tool schemas, then retrieve its proposed call.

3

Compare the call with your prediction.

Check the tool name and the requested clinic.

You have succeeded when the call is `get_clinic_status` with `clinic_id = KCARE-ADJ-01`.

Deterministic preview in mock mode. The notebook explains how to use Gemini for the mission.

Tell the model what the tool found.

1

Execute and trace the call: TODO 3–4.

Keep the tool, its arguments, its result and the execution status.

2

Update the history: TODO 5–6.

Add the model's proposed call, followed by the result returned by the tool.

3

Read the first trace entry.

Find the retrieved temperature and the call that produced it.

The next turn must receive the actual reading. The trace lets you find it again.

Allow the next step or stop the mission.

1

Check the evidence: TODO 7–8.

The reading must match; approval must refer to the right incident and action.

2

Block repeated calls: TODO 9.

An identical call that already ran must trigger a controlled stop.

3

Run the mission, then a failure case.

Find the trace evidence that permits the conclusion or requires a stop.

For the critical mission, all five tools are traced and the simulated operator's approval is recorded.

The model says “resolved.” Can we accept its answer?

The proposed answer

“Everything is safe; incident resolved.”

The model has read the measurement, procedure and risk level.

What the trace shows

The risk requires review.

No incident created.

No approval recorded.

Python blocks the conclusion: stopped / review_required. Required approval is missing.

This deliberately incorrect answer comes from the deterministic failure simulator.

A test passes when the expected behavior is respected.

Scenario	Expected result	Solution	Test
Full critical alert	human_approved	human_approved	PASS
No investigation	blocked	blocked	PASS
Missing approval	review_required	review_required	PASS
Repeated identical call	blocked	blocked	PASS

Stopping can be the correct result: tests also check when the agent should refuse to continue.

Solution excerpts: ten deterministic scenarios, including seven deliberate failures. No API calls.

Test each safeguard against a specific case.

1

Complete TODO 10.

A scenario passes only when every check is true.

2

Run the ten scenarios.

Compare the normal path and deliberate failures with their expected results.

3

Explain one result to your partner.

Which failure did you trigger? Which check caught it?

Once the mission is approved and all ten evaluations pass, the JSON dossier becomes available.

The five minutes include reading the results. Evaluations stay deterministic.

The test must notice when a safeguard is removed.

Without the repetition guard

The model can request the same tool again. The repeat case no longer matches the expected behavior, so its test fails.

With the guard restored

Python recognizes the already executed call and stops the loop. The same test passes because the repeated call is blocked.

10/10 covers these ten scenarios. A new risk needs a new test case.

A dossier you can explain and inspect.

INCIDENT

INC-001

TOOL CALLS

5

SIMULATED OPERATOR

APPROVED

EVALUATIONS

10 / 10

```
{
  "alert_id": "CC-204",
  "execution_evidence": [{
    "tool": "get_clinic_status",
    "status": "success"
  }]
}
```

The file contains readings, tool calls and results, the simulated decision and the evaluations. Your chosen failure experiment is included too.

Pick a decision in the dossier and find the evidence that justifies it.

Shortened excerpt from the solution dossier in mock mode. The file records the exercise; no physical action occurs.

KEEP GOING

Use the code to add your own scenario.

[Workshop repository · French and English notebooks](#)

Open Colab, inspect the solution and download the materials.

[Anthropic · Building effective agents](#)

Further reading on workflows and agents.

[Google · Gemini API rate limits](#)

Check the limits before planning concurrent model calls.

THANK YOU

Which rule would you test in your own field?

Name an action, then a case where your agent should stop.

Example: prepare a refund, then block its execution when required approval is missing.



[Code and notebooks](#)

[Scan to open the repository](#)

BOSSA Chabel · Software Engineer · ENSGMM

github.com/chabelbossa · linkedin.com/in/chabel-holdo-bossa